МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Санкт-Петербургский национальный исследовательский университет информационных технологий, механики и оптики»

ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ

Лабораторная работа №4

по дисциплине

«Распределенные системы хранения данных»

Вариант 389123

Выполнил:
Хасаншин Марат Айратович P3333
Преподаватель:
Николаев Владимир Вячеславович

Санкт-Петербург

~ 2025 ~

Задание **3**
Этап 0. Настройка ВМ **4**
Этап 1. Конфигурация **5**
Задание 5
Конфигурация узла A 5
Конфигурация узла B 6
Настройка узла pgbouncer 8
Этап 2. Симуляция и обработка сбоя **9**
Задание 9
2.1 Подготовка 9
2.2 Сбой 10
2.3 Обработка 11
Этап 3. Восстановление **12**
Задание 12
Выполнение 13
Код программы **15**
Вывод **16**

Задание

Внимание! У разных вариантов разный текст задания!

Цель работы - ознакомиться с методами и средствами построения отказоустойчивых решений на базе СУБД Postgres; получить практические навыки восстановления работы системы после отказа.

Работа рассчитана на двух человек и выполняется в три этапа: настройка, симуляция и обработка сбоя, восстановление.

Требования к выполнению работы

- В качестве хостов использовать одинаковые виртуальные машины.
- В первую очередь необходимо обеспечить сетевую связность между ВМ.
- Для подключения к СУБД (например, через psql), использовать отдельную виртуальную или физическую машину.
- Демонстрировать наполнение базы и доступ на запись на примере **не менее, чем двух** таблиц, столбцов, строк, транзакций и клиентских сессий.

Этап 1. Конфигурация

Развернуть postgres на двух узлах в режиме балансировки нагрузки. Использовать pgbouncer. Продемонстрировать обработку транзакций обоими серверами.

Этап 2. Симуляция и обработка сбоя

2.1 Подготовка:

- Установить несколько клиентских подключений к СУБД.
- Продемонстрировать состояние данных и работу клиентов в режиме чтение/запись.

2.2 Сбой:

Симулировать переполнение дискового пространства на основном узле - заполнить всё свободное пространство раздела с PGDATA “мусорными” файлами.

2.3 Обработка:

- Найти и продемонстрировать в логах релевантные сообщения об ошибках.
- Выполнить переключение (failover) на резервный сервер.
- Продемонстрировать состояние данных и работу клиентов в режиме чтение/запись.

Восстановление

- Восстановить работу основного узла - откатить действие, выполненное с виртуальной машиной на этапе 2.2.
- Актуализировать состояние базы на основном узле - накатить все изменения данных, выполненные на этапе 2.3.
- Восстановить исправную работу узлов в исходной конфигурации (в соответствии с этапом 1).
- Продемонстрировать состояние данных и работу клиентов в режиме чтение/запись.

Этап 0. Настройка ВМ

Порты для подключения: 2222, 2224
Имя пользователя: admin
Пароль: 1234

Параметры локальной сети ВМ:
- узел pgbouncer: 192.168.100.10
- узел A: 192.168.100.11
- узел B: 192.168.100.12

Порты, на которых работают postgresql
- Узел A: 5432
- Узел B: 5432

```
sudo apt update
sudo apt install -y wget gnupg lsb-release

wget --quiet -O - https://www.postgresql.org/media/keys/ACCC4CF8.asc | sudo
gpg --dearmor -o /usr/share/keyrings/postgresql.gpg

echo "deb [signed-by=/usr/share/keyrings/postgresql.gpg]
http://apt.postgresql.org/pub/repos/apt/ $(lsb_release -cs)-pgdg main" | \
  sudo tee /etc/apt/sources.list.d/pgdg.list

sudo apt update
sudo apt install -y postgresql postgresql-contrib

sudo systemctl status postgresql
```

Настройка сетевого взаимодействия:

```
> sudo apt update && sudo apt install nano iputils-ping -y
> sudo nano /etc/netplan/01-netcfg.yaml
network:
  version: 2
  ethernets:
    enp0s3:               # NAT-интерфейс
      dhcp4: true         # Оставляем по DHCP
    enp0s8:               # Внутренняя сеть
      dhcp4: no
      addresses:
        - 192.168.100.11/24
> sudo netplan apply
```

Этап 1. Конфигурация

Задание

Развернуть postgres на двух узлах в режиме балансировки нагрузки. Использовать pgbouncer. Продемонстрировать обработку транзакций обоими серверами.

Конфигурация узла A

/etc/postgresql/17/main/postgresql.conf

```
listen_addresses = '*'
port = 5432
wal_level = replica
max_wal_senders = 10
hot_standby = on
wal_keep_size = 256MB
archive_mode = on
archive_command = 'test ! -f /var/lib/postgresql/wal_archive/%f && cp %p
/var/lib/postgresql/wal_archive/%f’
```

/etc/postgresql/17/main/pg_hba.conf

```
# Database administrative login by Unix domain socket
local   replication     postgres                                peer
local   replication     replicator                              scram-sha-256

# "local" is for Unix domain socket connections only
local   all             all                                     peer
# IPv4 local connections:
host    all             all             127.0.0.1/32            scram-sha-256
host    replication     replicator      127.0.0.1/32            scram-sha-256
# IPv6 local connections:
host    all             all             ::1/128                 scram-sha-256
host    replication     replicator      ::1/128                 scram-sha-256

# Allow replication connections from localhost, by a user with the
# replication privilege

host    all             all             192.168.100.0/24        scram-sha-256
host    replication     replicator      192.168.100.0/24        scram-sha-256
host    all             pooler          192.168.100.10/32       scram-sha-256
```

```

```

Создадим роли для репликации и pgbouncer

```
psql -U postgres -c "
    CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD '1234';
    CREATE ROLE pooler WITH LOGIN PASSWORD '1234';
"
```

Перезапустим postgreSQL

```
sudo systemctl restart postgresql
ss -tulnp | grep 5432
```

```
admin@node1:~$ ss -tulnp | grep 5432
tcp   LISTEN 0      200              0.0.0.0:5432       0.0.0.0:*
tcp   LISTEN 0      200                 [::]:5432          [::]:*
```

```
SELECT pg_is_in_recovery();
```

```
postgres=# SELECT pg_is_in_recovery();
 pg_is_in_recovery
-------------------
 f
```

Конфигурация узла B

Остановим PostgreSQL и очистим каталог данных PostgreSQL

```
sudo systemctl stop postgresql
sudo rm -rf /var/lib/postgresql/17/main/*
```

Создадим базовую резервную копию с узла A на узел B

```
sudo -u postgres pg_basebackup -h 192.168.100.11 -p 5432 -U replicator -D
/var/lib/postgresql/17/main -Fp -Xs -P -R -v
```

Проверим параметры репликации /var/lib/postgresql/17/main/postgresql.auto.conf, а также добавим параметр application_name=standby_b

```
primary_conninfo = '
    user=replica_user
    password=strong_password
    channel_binding=prefer
    host=192.168.100.11
    port=5432
    sslmode=prefer
    sslnegotiation=postgres
    sslcompression=0
```

```
    sslcertmode=allow
    sslsni=1
    ssl_min_protocol_version=TLSv1.2
    gssencmode=prefer
    krbsrvname=postgres
    gssdelegation=0
    target_session_attrs=any
    load_balance_hosts=disable'
    application_name=standby_b
```

Создадим сигнальный файл, если он не был создан автоматически

```
sudo touch /var/lib/postgresql/17/main/standby.signal
```

Перезапускаем сервер на узле B

```
sudo systemctl start postgresql
```

```
user@rshd2:/var/lib/postgresql/17$ sudo tail -f /var/log/postgresql/postgresql-17-main.log
2025-06-05 13:12:40.504 UTC [2357] LOG:  listening on IPv6 address "::", port 5433
2025-06-05 13:12:40.509 UTC [2357] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5433"
2025-06-05 13:12:40.519 UTC [2360] LOG:  database system was interrupted; last known up at 2025-06-05 12:54:52 UTC
2025-06-05 13:12:41.351 UTC [2360] LOG:  starting backup recovery with redo LSN 0/2000028, checkpoint LSN 0/2000080, on timeline ID 1
2025-06-05 13:12:41.352 UTC [2360] LOG:  entering standby mode
2025-06-05 13:12:41.363 UTC [2360] LOG:  redo starts at 0/2000028
2025-06-05 13:12:41.367 UTC [2360] LOG:  completed backup recovery with redo LSN 0/2000028 and end LSN 0/2000120
2025-06-05 13:12:41.367 UTC [2360] LOG:  consistent recovery state reached at 0/2000120
2025-06-05 13:12:41.367 UTC [2357] LOG:  database system is ready to accept read-only connections
2025-06-05 13:12:41.400 UTC [2361] LOG:  started streaming WAL from primary at 0/3000000 on timeline 1
```

Лог содержит:

```
consistent recovery state reached at 0/2000120
database system is ready to accept read-only connections
started streaming WAL from primary at 0/3000000 on timeline 1
```

Значит всё отработало корректно

```
admin@node2:~$ sudo -u postgres psql -d postgres -c "SELECT pg_is_in_recovery();"
[sudo] password for admin:
 pg_is_in_recovery
-------------------
 t
(1 row)
```

Значит мы всё настроили корректно

Настройка узла pgbouncer

```
sudo apt install -y pgbouncer
```

demo_rw - используется для записи, балансировка отключена, потому что не поддерживается
demo_ro - чтение, с балансировакой round-robin

```
[databases]
```

```
demo_rw = host=192.168.100.11,192.168.100.12 port=5432 dbname=postgres
user=pooler password=1234 load_balance_hosts=disable
demo_ro = host=192.168.100.11,192.168.100.12 port=5432 dbname=postgres
user=pooler password=1234
[pgbouncer]
listen_addr       = 0.0.0.0
listen_port       = 6432

auth_type         = md5
auth_file         = /etc/pgbouncer/userlist.txt

pool_mode         = transaction
server_reset_query = DISCARD ALL

server_round_robin = 1

max_client_conn   = 200
default_pool_size = 1
server_lifetime   = 0

admin_users = pooler
stats_users = pooler

server_check_query = SELECT 1 WHERE pg_is_in_recovery() = false;
server_check_delay = 2
server_login_retry = 3
server_fast_close  = 1
```

```
sudo systemctl enable --now pgbouncer
```

```
/etc/pgbouncer/userlist.txt
"pooler" "md5f796b4808dde853e222502eab39cbfa7"
```

Этап 2. Симуляция и обработка сбоя

Задание

2.1 Подготовка:

- Установить несколько клиентских подключений к СУБД.
- Продемонстрировать состояние данных и работу клиентов в режиме чтение/запись.

2.2 Сбой:

Симулировать переполнение дискового пространства на основном узле - заполнить всё свободное пространство раздела с PGDATA “мусорными” файлами.

2.3 Обработка:

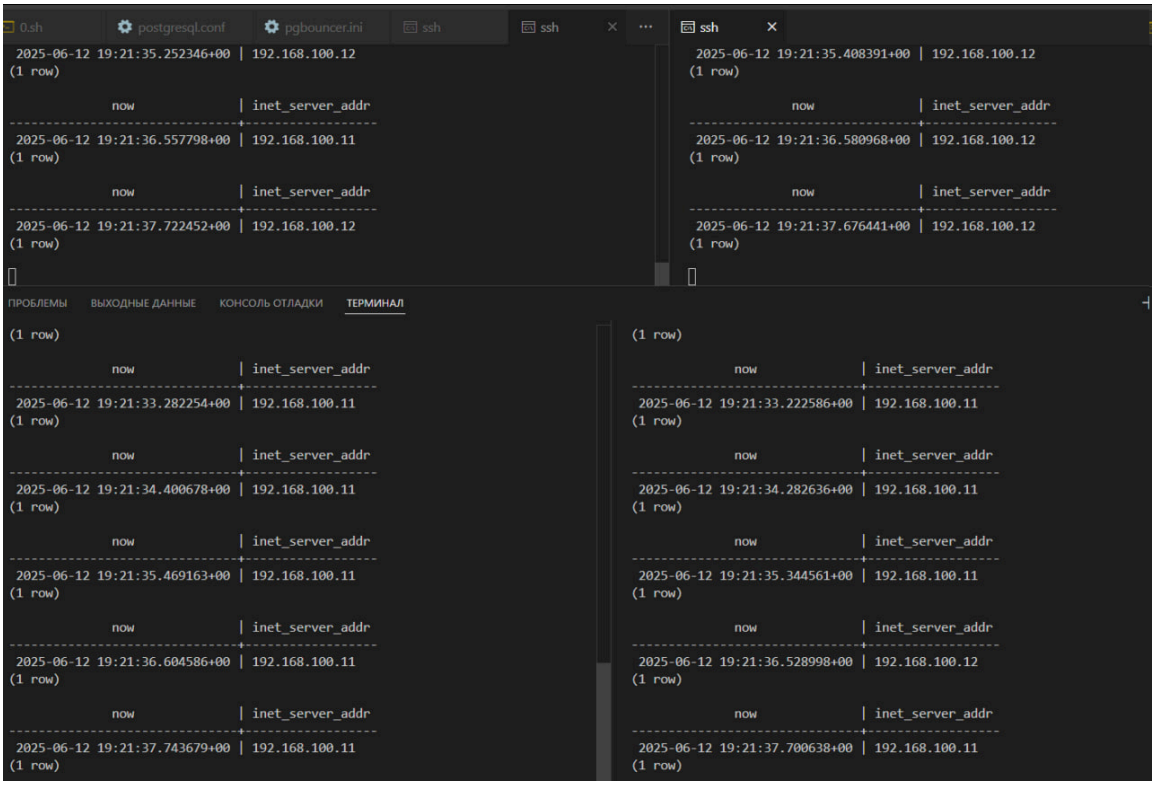
- Найти и продемонстрировать в логах релевантные сообщения об ошибках.
- Выполнить переключение (failover) на резервный сервер.
- Продемонстрировать состояние данных и работу клиентов в режиме чтение/запись.

Этап 2.1 Подготовка

В 4 терминалах запускаю программу на чтение

```
while true; do
  PGPASSWORD=1234 psql -h 127.0.0.1 -p 6432 -U pooler demo_ro \
    -c "SELECT now(), inet_server_addr();"
  sleep 1
done
```

Чтение происходит поочередно из 1 и 2 узла



При запуске программы для записи

```
for i in {1..6}; do
  PGPASSWORD=1234 psql -h 127.0.0.1 -p 6432 -U pooler -d demo_rw -Atc \
    "insert into lb_test(node) values (inet_server_addr()) returning
id,node,ts;";
done
```

Запись происходит только в основной узел

```
admin@pgbouncer:~$ for i in {1..6}; do   PGPASSWORD=1234 psql -h 127.0.0.1 -p 6432 -U pooler -d demo_rw -Atc    "insert into lb_test(node) values (inet_server_addr
()) returning id,node,ts;"; done
56|192.168.100.11/32|2025-06-12 21:43:14.136546+00
INSERT 0 1
57|192.168.100.11/32|2025-06-12 21:43:14.187376+00
INSERT 0 1
58|192.168.100.11/32|2025-06-12 21:43:14.248511+00
INSERT 0 1
59|192.168.100.11/32|2025-06-12 21:43:14.309623+00
INSERT 0 1
60|192.168.100.11/32|2025-06-12 21:43:14.36439+00
INSERT 0 1
61|192.168.100.11/32|2025-06-12 21:43:14.421302+00
INSERT 0 1
```

Этап 2.2 Сбой

```
admin@node1:~$ sudo bash fill.sh
Свободного места до заполнения:
4560244
Заполнение диска мусорными файлами в /var/lib/postgresql/17/main/filldisk_trash...
Свободное место закончилось.
Создано 45 файлов, каждый по 100 MB.
Свободного места после заполнения:
    0
```

Этап 2.3 Обработка

```
postgres=# insert into lb_test(node) values (inet_server_addr()) returning id, node, ts;
PANIC:  could not write to file "pg_wal/xlogtemp.4567": No space left on device
server closed the connection unexpectedly
        This probably means the server terminated abnormally
        before or while processing the request.
The connection to the server was lost. Attempting reset: Failed.
The connection to the server was lost. Attempting reset: Failed.
```

```
sudo -u postgres pg_ctlcluster 17 main promote
```

```
2025-06-12 22:29:08.342 UTC [2621] LOG:  received promote request
2025-06-12 22:29:08.342 UTC [2621] LOG:  redo done at 0/500FDE0 system usage:
CPU: user: 0.00 s, system: 0.02 s, elapsed: 4904.79 s
2025-06-12 22:29:08.398 UTC [2618] LOG:  database system is ready to accept
connections
```

```
admin@pgbouncer:~$ while true; do   PGPASSWORD=1234 psql -h 127.0.0.1 -p 6432
-U pooler demo_ro     -c "SELECT now(), inet_server_addr();";   sleep 1; done
              now              | inet_server_addr
-------------------------------+------------------
 2025-06-12 22:32:45.513598+00 | 192.168.100.12
(1 row)

              now              | inet_server_addr
-------------------------------+------------------
 2025-06-12 22:32:49.655371+00 | 192.168.100.12
(1 row)

              now              | inet_server_addr
-------------------------------+------------------
 2025-06-12 22:32:53.986167+00 | 192.168.100.12
(1 row)
```

```
admin@node2:~$ sudo -u postgres psql
psql (17.5 (Ubuntu 17.5-1.pgdg24.04+1))
Type "help" for help.

postgres=# create table test1 (
postgres(# id INT PRIMARY KEY,
postgres(# name VARCHAR(100) NOT NULL);
CREATE TABLE
postgres=# INSERT INTO test1(id, name) VALUES (1, 'Иван'), (2, 'Егор'),(3, 'Марат');
INSERT 0 3
postgres=# SELECT * FROM test1;
 id | name
----+-------
  1 | Иван
  2 | Егор
  3 | Марат
(3 rows)
```

Этап 3. Восстановление

Задание

- Восстановить работу основного узла - откатить действие, выполненное с виртуальной машиной на этапе 2.2.
- Актуализировать состояние базы на основном узле - накатить все изменения данных, выполненные на этапе 2.3.
- Восстановить исправную работу узлов в исходной конфигурации (в соответствии с этапом 1).
- Продемонстрировать состояние данных и работу клиентов в режиме чтение/запись.

Выполнение

```
admin@node1:~$ sudo bash clear.sh
Свободного места до очистки:
    0
Удаление мусорных файлов из /var/lib/postgresql/17/main/filldisk_trash...
Очистка завершена.
Свободного места после очистки:
4575364
admin@node1:~$ sudo -u postgres psql
psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL
" failed: No such file or directory
        Is the server running locally and accepting connections on that s
?
admin@node1:~$ sudo systemctl start postgresql@17-main
admin@node1:~$ sudo -u postgres psql
psql (17.5 (Ubuntu 17.5-1.pgdg24.04+1))
Type "help" for help.

postgres=#
```

До актуализации данных:

```
postgres=# \d
               List of relations
 Schema |       Name       |   Type   |  Owner
--------+------------------+----------+----------
 public | lb_test          | table    | postgres
 public | lb_test_id_seq   | sequence | postgres
 public | pgbench_accounts | table    | postgres
 public | pgbench_branches | table    | postgres
 public | pgbench_history  | table    | postgres
 public | pgbench_tellers  | table    | postgres
(6 rows)
```

```
sudo -u postgres pg_basebackup \
      -R                \
      -X stream         \
      -h 192.168.100.12  \
      -U replicator     \
      -D /var/lib/postgresql/17/main \
      -vP
```

```
admin@node1:~$ sudo -u postgres pg_basebackup -R -X stream -C -S node1_slot   -h 192.168.100.12 -U replic
ator   -D /var/lib/postgresql/17/main -vP
Password:
pg_basebackup: initiating base backup, waiting for checkpoint to complete
pg_basebackup: checkpoint completed
pg_basebackup: write-ahead log start point: 0/7000028 on timeline 2
pg_basebackup: starting background WAL receiver
pg_basebackup: created replication slot "node1_slot"
40101/40101 kB (100%), 1/1 tablespace
pg_basebackup: write-ahead log end point: 0/7000120
pg_basebackup: waiting for background process to finish streaming ...
pg_basebackup: syncing data to disk ...
pg_basebackup: renaming backup_manifest.tmp to backup_manifest
pg_basebackup: base backup completed
admin@node1:~$ sudo systemctl start postgresql@17-main
admin@node1:~$ sudo -u postgres psql -c "SELECT pg_is_in_recovery();"   # должно вернуть t
 pg_is_in_recovery
-------------------
 t
```

Актуальные данные:

```
postgres=# \d
                List of relations
 Schema |       Name        |   Type   |  Owner
--------+-------------------+----------+----------
 public | lb_test           | table    | postgres
 public | lb_test_id_seq    | sequence | postgres
 public | pgbench_accounts  | table    | postgres
 public | pgbench_branches  | table    | postgres
 public | pgbench_history   | table    | postgres
 public | pgbench_tellers   | table    | postgres
 public | test1             | table    | postgres
(7 rows)

postgres=# select * from test1;
 id | name
----+-------
  1 | Иван
  2 | Егор
  3 | Марат
(3 rows)
```

Возврат к исходной конфигурации:

```
sudo systemctl pg_ctlcluster 17 main promote
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
```

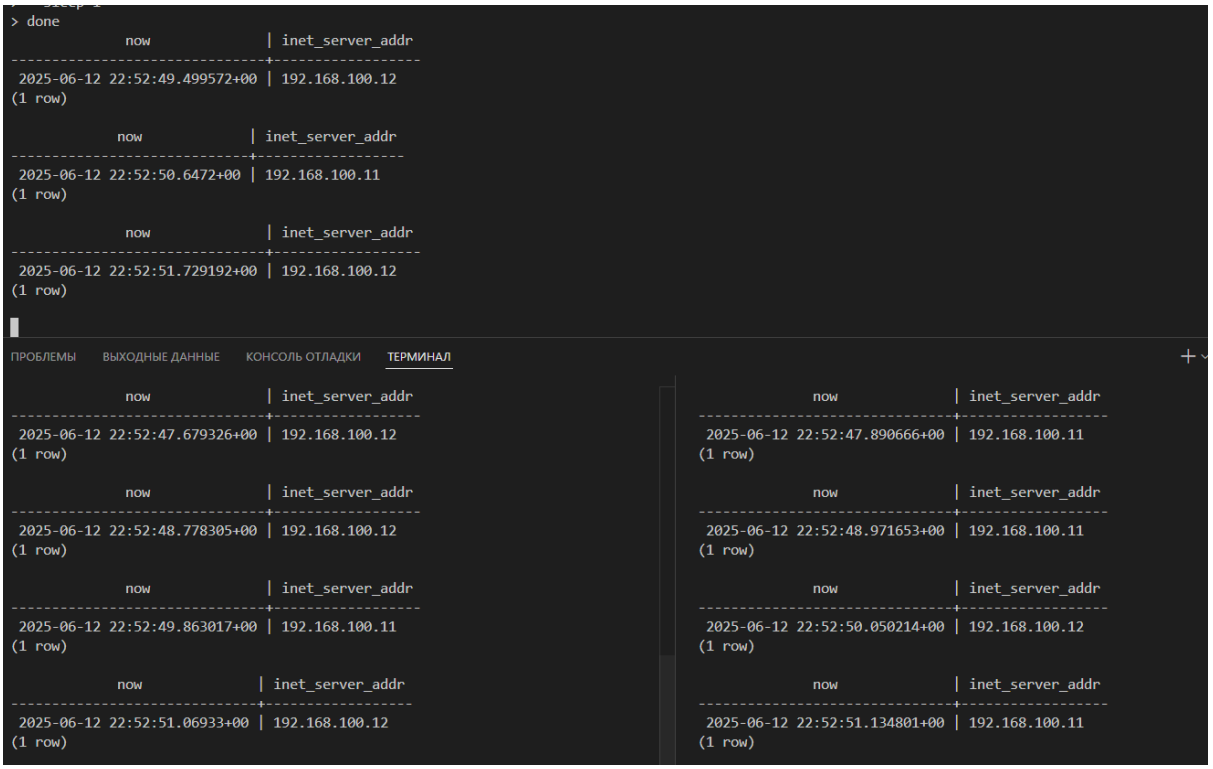
```
admin@node1:~$ sudo -u postgres pg_ctlcluster 17 main promote
admin@node1:~$ sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
 pg_is_in_recovery
-------------------
 f
(1 row)
```

```
sudo systemctl stop postgresql@17-main
sudo rm -rf /var/lib/postgresql/17/main

sudo -u postgres pg_basebackup \
>   -R \
>   -X stream \
>   -h 192.168.100.11 \
>   -U replicator \
>   -D /var/lib/postgresql/17/main \
>   -vP
sudo systemctl start postgresql@17-main
```

```
admin@node2:~$ sudo systemctl stop postgresql@17-main
admin@node2:~$ sudo -u postgres touch /var/lib/postgresql/17/main/standby.signal
admin@node2:~$ sudo systemctl start postgresql@17-main
admin@node2:~$ sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
 pg_is_in_recovery
-------------------
 t
```

Чтение:



sudo systemctl restart pgbouncer

